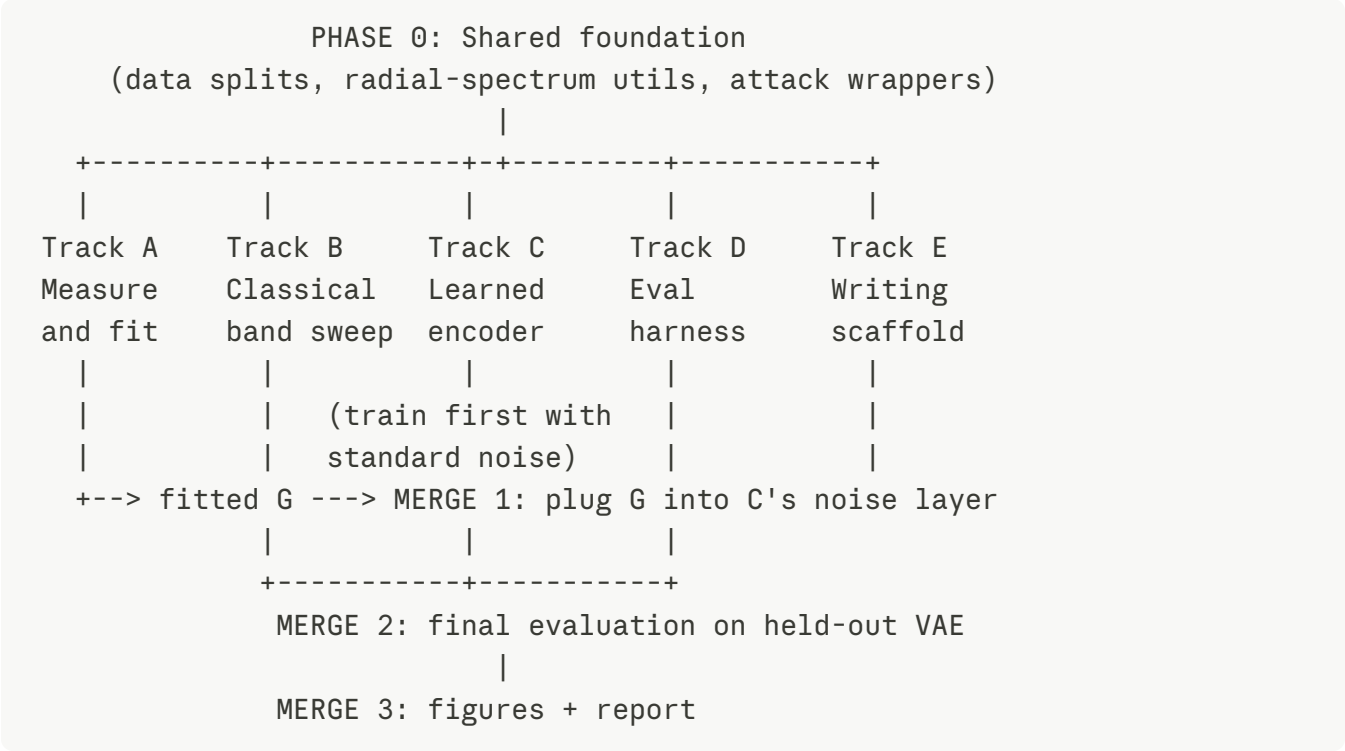
Project Guide: Spectral Band Allocation for Invisible Watermarks Under VAE Regeneration

Working companion to the proposal: what to build, what data to get, and how the pieces fit together.

1. How the work is organised

One short shared foundation (Phase 0), then five tracks that run side by side and meet at three merge points. Tracks talk to each other only through four small contracts (Section 4), so no track blocks another.



Working solo? Parallel then means interleaving: launch the long GPU jobs (Track A sweeps, Track C training) first, and write the other tracks' code while they run.

2. Data and models

2.1 Datasets

Dataset	Used for	How much	Source
COCO 2017 train	Training split (Track C); fit split (Tracks A, B)	10-20k images for training, 300 for fitting	cocodataset.org
COCO 2017 val	Held-out test split (all tracks)	500 images	cocodataset.org (val2017, 5k images, about 1 GB)
Kodak 24 (optional)	Quick visual and spectral sanity checks	24 images	r0k.us/graphics/kodak

- The full train2017 archive is about 18 GB. If that is too heavy, stream a subset from a COCO mirror on Hugging Face, or split val2017 as 4000 train / 300 fit / 500 test. A tiny encoder trains fine on that.
- Splits must never overlap. Save manifests (lists of file names) once and reuse them everywhere.
- Preprocess once: resize the shorter side to 256 (area or bicubic), center-crop to 256x256, save fit and test sets as PNG. Training uses random 256 crops (or 128 for speed) from images resized the same way.

2.2 Pretrained models (no training needed)

Model	Role	How to get it
CompressAI bmshj2018-factorized, quality 1-8	Primary Regen-VAE with a built-in strength knob (low quality = strong attack)	pip install compressai; compressai.zoo.bmshj2018_factorized(quality=q, pretrained=True)
CompressAI cheng2020-anchor (optional)	Second compressor-VAE for robustness of the fit	same zoo

Model	Role	How to get it
Stable Diffusion VAE (stabilityai/sd-vae-ft-mse)	Held-out VAE, never used to fit the surrogate	diffusers AutoencoderKL
LPIPS (AlexNet)	Perceptual metric	pip install lpips
TrustMark	External reference watermark	pip install trustmark

Fit / test split of attacks. Fit G and train the surrogate from the CompressAI family; evaluate the encoder on the SD VAE, which it has never seen. The SD VAE has no quality knob, so make strengths by adding Gaussian noise (std sigma) to the image before encoding, the noise-then-reconstruct pattern from the regeneration literature. Swapping the two roles is a cheap ablation.

2.3 Software

torch, torchvision, numpy, scipy, scikit-image, lpips, compressai, diffusers (+ transformers, accelerate), kornia, matplotlib, pandas, tqdm. Run CPU-bound work (FFT analysis, metrics, plots) locally and VAE sweeps and training on GPU. Every GPU job writes results to CSV and saves checkpoints as it goes.

3. Repository layout

```
project/
  data/       manifests + 256px PNG fit/test sets
  core/       radial.py  bands.py  probes.py  attacks.py
  track_a/    measure.py  fit.py
  track_b/    sweep.py
  track_c/    model.py  noise_layers.py  train.py
  track_d/    metrics.py  harness.py  plots.py
  results/    tidy CSVs, one row per image x condition
  figures/    report/
```

4. Shared contracts (what lets tracks run independently)

1. **Attack:** attack(x, strength) -> x_attacked. x is a float tensor in [0,1], shape B x 3 x 256 x 256. Strength comes from a named list per attack family. Attacks are deterministic: for stochastic ones, fix the seed and use the same seed for A(x) and A(x_w).
2. **Watermarker:** embed(x, msg, psnr_db) -> x_w and decode(y) -> bit probabilities. The classical and learned systems both implement it.
3. **Results CSV schema:** method, band, attack, strength, psnr_target, psnr_actual, image_id, bit_acc, lpips, ssim.
4. **Fitted-G file (JSON):** attack, strength, A, alpha, N_eff, fit_r2, f_c_pred, f_c_meas. Track A writes it, Track C reads it.

5. Phase 0: Shared foundation

0.1 Environment and data

- ☐ Install libraries; confirm GPU is visible.
- ☐ Download COCO val2017 (and a train subset); build train / fit / test manifests.
- ☐ Run the preprocessing script; spot-check a few PNGs.

0.2 Radial spectrum utilities (core/radial.py, core/bands.py)

How: take the luminance channel Y, compute fft2 with norm='ortho' (this convention matters later), then fftshift. Build a frequency grid in cycles per pixel (0 to 0.5 along each axis) and give every frequency its distance from the center:

```
r = \sqrt{f_x^2 + f_y^2}
```

Radial profile = the mean power inside each of 32 equal-width radius bins from 0 to 0.5. Band masks = 8 disjoint annuli over [0, 0.5] (pick linear or log spacing once and keep it).

- ☐ radial_profile(power2d) and band_masks(n_bands).
- ☐ Test: white noise gives a flat profile; a pure sinusoid at a known frequency spikes in the right bin.

0.3 Probes (core/probes.py)

How: white Gaussian noise, then FFT, keep one annulus (symmetric mask), inverse FFT, normalise to unit RMS, scale it, and add it to the image. The scale comes from the PSNR you want, with images on the 0-255 scale:

```
a = 255 \cdot 10^{-\mathrm{PSNR}/20}
```

Here a is the RMS of the probe. Round to 8-bit afterwards, since real images are 8-bit.

- ☐ Test: PSNR of x vs x + probe matches the target within 0.1 dB after rounding.
- ☐ Test: the probe's spectrum lives only inside its annulus.

0.4 Attack wrappers (core/attacks.py)

- ☐ CompressAI wrapper, strengths = qualities 1-8.
- ☐ SD VAE wrapper (encode, take posterior mean, decode) with pre-noise sigma as strength; pick 4-5 values from no effect to clearly strong.
- ☐ Secondary attacks: Gaussian noise, blur, JPEG.
- ☐ Test: PSNR(x, A(x)) falls monotonically with strength; no NaNs; images look the same to the eye.
- ☐ Save a table of PSNR(x, A(x)) per strength; it anchors what strength means in the report.

Gate G0: one script loads an image, adds a probe, attacks it, plots the radial profile of the difference. When that plot looks sensible, start the tracks.

6. Track A: Measure and fit (the signal-processing core)

A1 Measure the transfer function

For every attack x strength x probe annulus x fit image: compute delta_in = x_w - x and delta_out = A(x_w) - A(x) (same seed, luminance, after rounding). Compute the radial energy of each. Then

The transfer function in each band is the square root of the average energy ratio:

```
H(\text{band}) = \sqrt{\operatorname{mean}_{\text{images}}
\frac{E_{\text{out}}}{E_{\text{in}}}}
```

In words: how much of the watermark's energy in that band survives the attack.

This isolates the response to the watermark itself, nonlinearity included.

- ☐ Run sweeps at PSNR 35, 40, 45; save to CSV. If H changes with amplitude, the attack is not linear at watermark scale, which is worth reporting.
- ☐ Plot H vs frequency, one curve per strength.

- ☐ Optional: a leakage matrix (probe band vs output band) to show energy moving across bands.
- ☐ Sanity: H close to 1 in the lowest bands at mild strength, falling at high frequency, cutoff moving lower as strength grows. If not, suspect a shift or padding misalignment or a colour-space mistake before suspecting a finding.

A2 Fit the image spectrum

For each fit image, compute the radial power profile P(f). Fit a straight line in log-log space over about f in [0.02, 0.4] (skip DC and near-Nyquist) by least squares:

```
\log P(f) = \log A - \alpha \, \log f
```

Natural-image power spectra typically give α near 2; record the distribution.

- ☐ Fit A and alpha per image and as a dataset average.

A3 Fit the Wiener model

For each strength, fit the single number N_eff in the Wiener gain below by minimising the squared error to the measured H over bands (scipy least_squares, parameter in log space):

```
G(f) = \frac{P(f)}{P(f) + N_{\text{eff}}}
```

In words: where the image has far more power than the noise, G is close to 1 and the watermark passes; where noise dominates, G falls toward 0. Define the cutoff as the frequency where H crosses 0.5. The model predicts it, because G equals 0.5 exactly when P equals N_eff:

```
f_c = \left( \frac{A}{N_{\text{eff}}} \right)^{1/\alpha}
```

Two non-circular tests make this a real prediction rather than a curve fit:

1. **Held-out strengths.** Fit N_eff at a subset of strengths, interpolate to the others, and compare predicted vs measured cutoff there.
2. **Theory check on the SD VAE.** With orthonormal FFT, noise of standard deviation σ added before encoding has a flat spectrum of height σ², so N_eff should scale roughly linearly with σ². Plot N_eff against σ².

- ☐ Table of predicted vs measured f_c; scatter plot with y = x line; fit R^2.
- ☐ Export fitted-G JSON (Merge 1 input).

- ☐ If the fit is poor, generalise to G = (P / (P + N))^beta or add a floor for leakage, and report the systematic deviation as a result.

7. Track B: Classical band sweep

B1 Spread-spectrum watermark

For each of k = 32 bits, build a carrier c_i: band-limited white noise (same construction as a probe), unit RMS, seeded by (key, bit index, band). The message signal adds the carriers up, with a plus or minus sign per bit:

```
s = \sum_{i=1}^{k} (2 b_i - 1)\, c_i
```

Normalise s to the RMS that gives the target PSNR and add the same pattern to all RGB channels (a luminance change). Decode blind by correlating the received image y with each carrier:

```
\hat{b}_i = \begin{cases} 1 & \text{if } \langle y - \bar{y},\, c_i \rangle > 0 \\ 0 & \text{otherwise} \end{cases}
```

Expect cover interference: natural images carry most energy at low frequency, so low-band carriers face more interference even with no attack. Always record a no-attack row per band so the attack effect is separable from this ceiling.

- ☐ Implement embed / decode behind the watermarker contract.
- ☐ Test: no attack, 40 dB, bit accuracy near 100% in mid bands.

B2 Sweep

- ☐ Bands (8) x strengths x PSNR targets (e.g. 35, 38, 41, 44) x 300 images, all logged with lpips and ssim.
- ☐ Detection: threshold tau = smallest bit-agreement count whose binomial tail, scipy.stats.binom.sf under p = 0.5 with n = k, is at most the target FPR. TPR = fraction of attacked watermarked images at or above tau. Check the null empirically on unwatermarked images.
- ☐ Heatmaps per PSNR: x = band, y = strength, colour = bit accuracy and TPR.

B3 Connect to the model

- ☐ Overlay the fitted f_c(strength) curve on the heatmap and mark the best band per strength (tests H2).

- ☐ Compare LPIPS across bands at equal PSNR to show which bands cost more perceptually.

8. Track C: Learned encoder with band allocation

C1 Architecture

- Encoder: message (k bits) -> linear layer -> spatial map, concatenated with image features -> a few conv layers -> residual (3 channels, or 1 channel added to Y).
- Decoder: 6-8 strided conv layers with BatchNorm, global average pool, linear to k logits (ResNet-18 scale at most).
- Start with k = 16 for debugging, then k = 32.

C2 Band-allocation layer (the contribution)

How: take the FFT of the residual, split it with the 8 disjoint band masks, normalise each band to unit RMS, then scale each band by its share of the energy budget. The shares come from learnable numbers θ_b passed through a softmax, so they are positive and add up to 1:

```
g_b = \frac{e^{\theta_b}}{\sum_j e^{\theta_j}}, \qquad \text{band } b \text{
is scaled by } \sqrt{g_b \, E_{\text{total}}}
```

Because bands are disjoint, total energy equals E_total exactly, set from the target PSNR (e.g. 40 dB). The network therefore cannot cheat by turning up the amplitude: PSNR is fixed by construction, and g_b is literally the fraction of energy in band b.

- ☐ Implement the layer; test that residual RMS equals target and that g sums to 1.
- ☐ Controls: same model with theta frozen to concentrate energy in low (bands 0-2), mid (3-5), or high (6-7).

C3 Noise layer (modular, plug-in)

A list of differentiable callables sampled per batch: identity, Gaussian noise, blur, differentiable JPEG approximation (recent kornia versions include one; otherwise use DCT quantisation with straight-through rounding or skip it), and the **spectral-transfer surrogate**.

Surrogate: load the fitted-G JSON, sample a strength, and build G on the rfft2 grid from (A, α, N_eff). The attacked image is the cover plus the watermark residual passed through G in the Fourier domain:

```
x_{\text{att}} = x + \mathcal{F}^{-1}\big( G \cdot \mathcal{F}(x_w - x) \big)
```

This is the linearised model: the cover passes through untouched and only the watermark is filtered. Until Merge 1, this slot returns identity. A whole-image variant (filtering x_w itself) is a cheap ablation.

- ☐ Unit-test each layer: shapes, gradients flow, strength ordering sensible.
- ☐ Add 8-bit rounding with a straight-through estimator at the end of the encoder.

C4 Training recipe

- Starting points: Adam lr 1e-3 with cosine decay, batch 32, mixed precision, loss = BCE (weight 1) + small LPIPS and MSE terms.
- Curriculum: identity and light noise first so the bits get learned, then the full mixture.
- ☐ Sanity run: overfit about 100 images with identity noise; bit accuracy should reach near 100% quickly. If it stalls near 50%, check message injection and loss wiring before tuning anything.
- ☐ Log bit accuracy per noise type separately, not just the mean.
- ☐ Train the main model, the three fixed-band controls, and per-strength models (or condition the gains on strength) to get the gains-vs-strength figure.
- ☐ Save checkpoints and the learned g after every epoch.

C5 After Merge 1

- ☐ Retrain with the surrogate in the mixture. Compare against the same model without it on the held-out SD VAE; that gap is the evidence that the fitted surrogate helps.

9. Track D: Evaluation harness

- ☐ metrics.py: PSNR (8-bit RGB), SSIM, LPIPS, bit accuracy.
- ☐ Binomial threshold and TPR@FPR function (shared with Track B).
- ☐ Empirical null check: decode unwatermarked, attacked images and confirm bit agreement averages near 50% with no strong per-bit bias.
- ☐ runner: takes any watermarker and an attack list, writes the results CSV schema.
- ☐ plots.py: heatmap, line plot, bar chart, all reading from CSVs only.
- ☐ TrustMark reference run through the same runner and attacks.

Reporting rule: with 500 test images the empirical FPR resolution is about 0.2%, so TPR@1%FPR is the primary metric and TPR@0.1%FPR is computed from the binomial null. State this explicitly.

10. Track E: Writing scaffold

- ☐ Report skeleton: Introduction, Background, Method (spectral model, watermark systems), Experiments, Results, Discussion, Limitations.
- ☐ Claims log: one row per hypothesis (H1-H3) with the figure that tests it and its status (supported, partly, not supported).
- ☐ Write the method text as each track finishes, while it is fresh.
- ☐ Leave placeholder figure slots; fill them as CSVs arrive.

11. Merge points

Merge 1 (A -> C). Track A exports the fitted-G JSON; Track C switches the surrogate slot on and retrains.

Merge 2 (A + B + C + D). Run every watermarker (classical sweep, learned model, three controls, TrustMark) through the same runner, on the same test images, against the same attacks, with the SD VAE as the held-out attack. Everything lands in one results folder.

Merge 3 (everything -> report). Generate figures from CSVs, update the claims log, finish the text.

12. Final deliverables checklist

- ☐ Fig 1: measured vs fitted transfer functions (Track A).
- ☐ Fig 2: predicted vs measured cutoff, plus N_eff against σ^2 (Track A).
- ☐ Fig 3: band x strength heatmaps for the classical watermark with f_c overlay (Track B).
- ☐ Fig 4: learned band gains vs attack strength (Track C).
- ☐ Table 1: learned vs fixed-band controls vs TrustMark at equal PSNR on the held-out VAE (Merge 2).
- ☐ Table 2: ablation, with vs without the fitted surrogate (Track C).
- ☐ Published W-Bench numbers for StegaStamp, MBRS and VINE cited as external context.

13. Failure modes and fallbacks

- **Fit is poor:** try the exponent or floor extension in A3; report the deviation.
- **Learned encoder does not beat the best fixed band:** that is a valid finding for H3, as long as measurement and fit are solid. Check the controls were trained as long as the main model.
- **Bits stuck near 50%:** reduce noise, run the identity-noise overfit test, verify the energy constraint is not shrinking the residual to nothing.
- **Decoder cheats via clipping or quantisation:** train with 8-bit rounding and evaluate on saved PNGs.
- **Tracks disagree** (for example the surrogate helps in training but not on the SD VAE): look at the Track A leakage matrix and the amplitude-dependence plot for where the linear model breaks.

14. Working with an AI coding assistant

- Write the interfaces (Section 4) and the unit tests first; have the assistant fill in implementations against them.
- Do not move past Gate G0 until the sanity tests pass. Known-answer tests (white noise, a sinusoid, PSNR targets) catch most silent bugs.
- Make sure you can explain the radial profile, the probe construction, and the Wiener fit yourself; these are what you will be asked about.
- Keep each notebook short and single-purpose, writing outputs to disk.